EXP NO 1:

Aim:

To partition the given sales price data into three bins using Equal-frequency (equi-depth) partitioning, Equal-width partitioning, Clustering-based partitioning and to implement the same using R programming.

ALGORITHM

Step 1: Initialize the sorted dataset: 5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215.

Step 2: Apply Equal-Frequency Partitioning by dividing the dataset into 3 bins with 4 elements in each bin.

Step 3: Apply Equal-Width Partitioning by calculating bin width using (maximum value − minimum value) divided by number of bins and assign data accordingly.

Step 4: Apply Clustering-Based Partitioning by performing k-means clustering with k = 3 to group similar values.

Step 5: Display the final bins and clusters obtained from all partitioning methods.

CODE:

```
data <- c(5,10,11,13,15,35,50,55,72,92,204,215)

bins_eq_freq <- split(data, cut(seq_along(data), 3, labels = FALSE))

print(bins_eq_freq)

bins_eq_width <- cut(data, breaks = 3, include.lowest = TRUE)

print(split(data, bins_eq_width))

set.seed(1)

k <- kmeans(data, centers = 3)

print(split(data, k$cluster))
```

OUTPUT:

```
Type 'q()' to quit R.

> data <- c(5,10,11,13,15,35,50,55,72,92,204,215)
>
> bins_eq_freq <- split(data, cut(seq_along(data), 3, labels = FALSE))
> print(bins_eq_freq)
$`1`
[1]  5 10 11 13

$`2`
[1] 15 35 50 55

$`3`
[1]  72  92 204 215

>
> bins_eq_width <- cut(data, breaks = 3, include.lowest = TRUE)
> print(split(data, bins_eq_width))
$`[4.79,75]`
[1]  5 10 11 13 15 35 50 55 72

$`(75,145]`
[1] 92

$`(145,215]`
[1] 204 215

>
> set.seed(1)
> k <- kmeans(data, centers = 3)
> print(split(data, k$cluster))
$`1`
[1] 204 215

$`2`
[1]  5 10 11 13 15 35

$`3`
[1] 50 55 72 92
>
```

EXP NO:02

Aim:

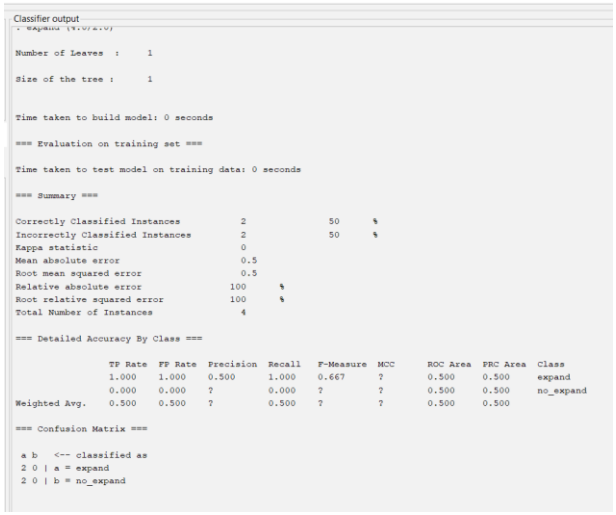
To analyze whether the factory should be expanded or not using a Decision Tree, based on expected revenues under good and bad economic conditions, and to identify the best decision using expected monetary value (EMV).

ALGORITHM

Identify decision alternatives: Expand factory, Do not expand factory.
Identify states of nature with probabilities: Good economy (0.45), Bad economy (0.55).
Assign payoffs for each decision–state combination.
Calculate EMV for each decision using EMV = Σ(Probability × Payoff).
Compare EMVs of all decisions.
Select the decision with the highest EMV.

CODE:

```
@relation Factory

@attribute Decision {Expand, NoExpand}

@attribute Economy {Good, Bad}

@attribute Profit numeric

@data

Expand, Good, 7

Expand, Good, 7

Expand, Good, 7

Expand, Good, 7

Expand, Bad, 3

Expand, Bad, 3

Expand, Bad, 3

Expand, Bad, 3

Expand, Bad, 3

NoExpand, Good, 4

NoExpand, Good, 4

NoExpand, Good, 4

NoExpand, Good, 4

NoExpand, Bad, 1.5

NoExpand, Bad, 1.5

NoExpand, Bad, 1.5
```



EXP NO:03

AIM

To apply the Apriori Algorithm on the given transaction database by assuming minimum support = 2 and implement it using the WEKA tool to generate frequent itemsets and association rules.

ALGORITHM (Apriori)

1. Read the transactional dataset.
2. Generate candidate 1-itemsets.
3. Count the support of each item.
4. Prune items whose support is less than minimum support (2).
5. Generate higher-order itemsets from frequent itemsets.
6. Repeat pruning until no new frequent itemsets are formed.
7. Generate association rules from frequent itemsets.

CODE:

@relation market_basket

@attribute Bread {t,f}

@attribute Peanuts {t,f}

@attribute Milk {t,f}

@attribute Fruit {t,f}

@attribute Jam {t,f}

@attribute Soda {t,f}

@attribute Chips {t,f}

@attribute Steak {t,f}

@attribute Cheese {t,f}

@attribute Yogurt {t,f}

@data

t,t,t,t,t,f,f,f,f,f

t,f,t,t,t,t,t,f,f,f

t,f,f,f,t,t,t,t,f,f

f,t,t,t,t,t,f,f,f,f

t,f,t,f,t,t,t,f,f,f

f,f,t,t,f,t,t,f,f,f

f,t,t,t,f,t,f,f,f,f

f,t,f,t,f,f,f,f,t,t

OUTPUT:

```
              Cheese
              Yogurt
=== Associator model (full training set) ===


Apriori
=======

Minimum support: 0.8 (6 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 4

Generated sets of large itemsets:

Size of set of large itemsets L(1): 6

Size of set of large itemsets L(2): 9

Size of set of large itemsets L(3): 5

Size of set of large itemsets L(4): 1

Best rules found:

 1. Yogurt=f 7 ==> Cheese=f 7    <conf:(1)> lift:(1.14) lev:(0.11) [0] conv:(0.88)
 2. Cheese=f 7 ==> Yogurt=f 7    <conf:(1)> lift:(1.14) lev:(0.11) [0] conv:(0.88)
 3. Milk=t 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 4. Milk=t 6 ==> Cheese=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 5. Milk=t 6 ==> Yogurt=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 6. Fruit=t 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 7. Soda=t 6 ==> Cheese=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 8. Soda=t 6 ==> Yogurt=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
 9. Steak=f Cheese=f 6 ==> Milk=t 6    <conf:(1)> lift:(1.33) lev:(0.19) [1] conv:(1.5)
10. Milk=t Cheese=f 6 ==> Steak=f 6    <conf:(1)> lift:(1.14) lev:(0.09) [0] conv:(0.75)
```

RESULT:

The Apriori algorithm was successfully applied using WEKA with minimum support of 2.

EXP NO:04

AIM

To normalize the given dataset using
(a) Min–Max normalization
(b) Z-score normalization using Mean Absolute Deviation (MAD)
(c) Decimal scaling normalization and implement the same using R programming.

ALGORITHM

Step 1:

Read the input dataset
200, 300, 400, 600, 1000

Step 2: Min–Max Normalization

- Find minimum and maximum values
- Apply formula:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Step 3: Z-score Normalization (MAD)

- Compute mean of the dataset
- Compute mean absolute deviation
- Apply formula:

$$z = \frac{x - \mu}{MAD}$$

Step 4: Decimal Scaling

- Find the maximum absolute value
- Determine number of digits j
- Divide all values by 10^j

Step 5:

Display the normalized values.

CODE:

```
# Given data
x <- c(200, 300, 400, 600, 1000)
# (a) Min-Max Normalization
min_max <- (x - min(x)) / (max(x) - min(x))
print(min_max)
# (b) Z-score Normalization using MAD
mean_x <- mean(x)
mad_x <- mean(abs(x - mean_x))
z_score_mad <- (x - mean_x) / mad_x
print(z_score_mad)
# (c) Decimal Scaling Normalization
j <- ceiling(log10(max(abs(x))))
```

decimal_scaled <- x / (10^j)

print(decimal_scaled)

```
$`3`
[1] 50 55 72 92

> # Given data
> x <- c(200, 300, 400, 600, 1000)
>
> # (a) Min-Max Normalization
> min_max <- (x - min(x)) / (max(x) - min(x))
> print(min_max)
[1] 0.000 0.125 0.250 0.500 1.000
>
> # (b) Z-score Normalization using MAD
> mean_x <- mean(x)
> mad_x <- mean(abs(x - mean_x))
> z_score_mad <- (x - mean_x) / mad_x
> print(z_score_mad)
[1] -1.2500000 -0.8333333 -0.4166667  0.4166667  2.0833333
>
> # (c) Decimal Scaling Normalization
> j <- ceiling(log10(max(abs(x))))
> decimal_scaled <- x / (10^j)
> print(decimal_scaled)
[1] 0.2 0.3 0.4 0.6 1.0
>
```

EXP NO:05

AIM

To analyze the relationship between Blood Pressure and Age of people affected by diabetes using the diabetes cs dataset, and to visualize the data using a scatter plot and a bar chart in R programming.

ALGORITHM

1. Load the diabetes dataset into R.
2. Extract the **Age** and **BloodPressure** attributes.
3. Plot a **scatter plot** to visualize the relationship between Age and Blood Pressure.
4. Plot a **bar chart** to represent Blood Pressure distribution across Age.
5. Interpret the visualizations.

CODE:

Set working directory

setwd("C:/Users/pathi/OneDrive/Desktop/DWDM")

Load the dataset

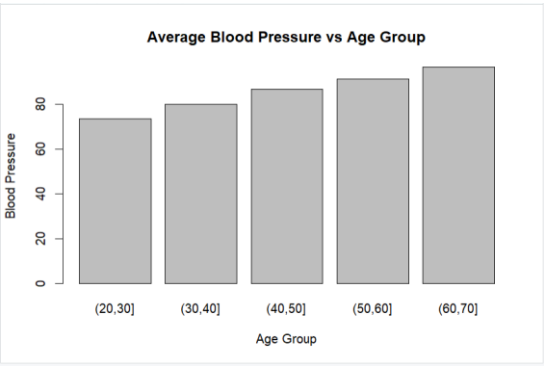
data <- read.csv("diabetes.csv")

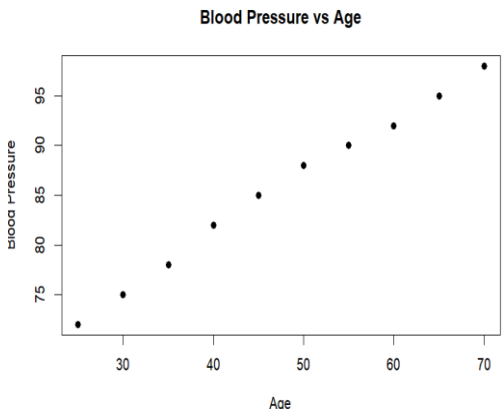
SCATTER PLOT: Blood Pressure vs Age

plot(data$Age, data$BloodPressure,

```
    main="Blood Pressure vs Age",
    xlab="Age",
    ylab="Blood Pressure",
    pch=19)

# Create Age groups for bar char
data$AgeGroup <- cut(data$Age,
                     breaks=c(20,30,40,50,60,70))

# Average Blood Pressure per Age group
avg_bp <- aggregate(BloodPressure ~ AgeGroup, data, mean)

# BAR CHART: Average Blood Pressure vs Age Group
barplot(avg_bp$BloodPressure,
        names.arg=avg_bp$AgeGroup,
        main="Average Blood Pressure vs Age Group",
        xlab="Age Group",
        ylab="Blood Pressure")
```

OUTPUT:





Result:

The relationship between **Age and Blood Pressure** in the diabetes dataset was successfully visualized using a scatter plot and a bar chart. The plots help in understanding how blood pressure varies with age among diabetic patients.

EXP NO 6:

AIM

To analyze the diabetes trend for different age groups using the dataset diabetes.csv by applying Linear Regression and Multiple Linear Regression and to study how age and other factors influence diabetes outcome using R programming

ALGORITHM

Step 1:

Load the diabetes dataset into R.

Step 2:

Identify relevant attributes:

- Age (independent variable)
- Outcome (dependent variable: 0 – Non-diabetic, 1 – Diabetic)

Step 3: Linear Regression

- Build a linear regression model using Age → Outcome
- Analyze the relationship between age and diabetes trend

Step 4: Multiple Regression

- Build a multiple regression model using Age, Glucose, BMI, BloodPressure → Outcome
- Analyze combined effect of multiple attributes on diabetes

Step 5:

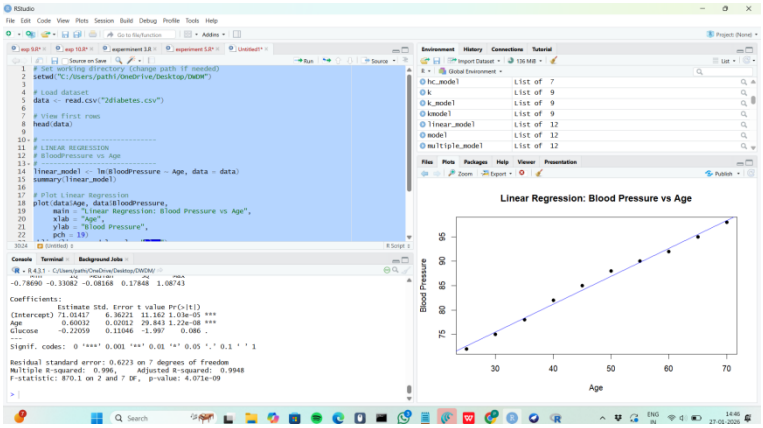
Display model summary and interpret results.

CODE:

```
# Set working directory (change path if needed)
setwd("C:/Users/pathi/OneDrive/Desktop/DWDM")


# Load dataset
data <- read.csv("2diabetes.csv")


# View first rows
head(data)


# ------------------------------
# LINEAR REGRESSION
# BloodPressure vs Age
# ------------------------------
linear_model <- lm(BloodPressure ~ Age, data = data)
```

```
summary(linear_model)


# Plot Linear Regression
plot(data$Age, data$BloodPressure,
     main = "Linear Regression: Blood Pressure vs Age",
     xlab = "Age",
     ylab = "Blood Pressure",
     pch = 19)
abline(linear_model, col = "blue")


# ------------------------------
# MULTIPLE REGRESSION
# BloodPressure vs Age + Glucose
# ------------------------------
multiple_model <- lm(BloodPressure ~ Age + Glucose, data = data)
summary(multiple_model)
```

OUTPUT:



RESULT:

The diabetes dataset analysis using linear and multiple regression shows that diabetes prevalence increases with age and is strongly influenced by factors such as glucose level, BMI, and blood pressure.

EXP NO 07

AIM

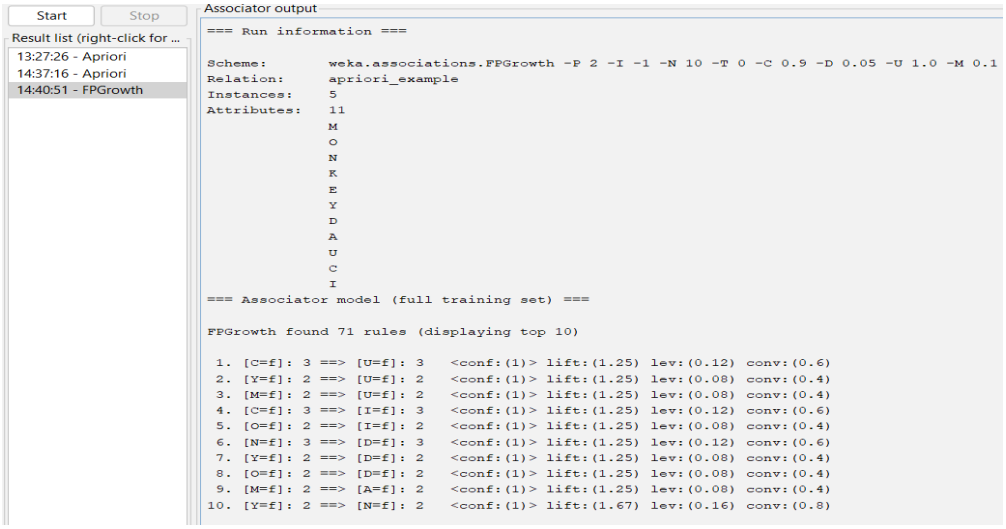
To generate frequent itemsets and association rules from the given transaction database using the Apriori algorithm with Minimum Support = 50% (2 transactions) Minimum Confidence = 80%, and to implement the same using the WEKA tool.

ALGORITHM (APRIORI)

1. Scan the transaction database.
2. Generate candidate 1-itemsets.
3. Count support for each item.
4. Prune itemsets whose support is less than minimum support.
5. Generate higher-order candidate itemsets from frequent itemsets.
6. Repeat pruning until no new frequent itemsets are generated.
7. Generate association rules from frequent itemsets.
8. Select rules whose confidence ≥ minimum confidence.

CODE:

```
@relation apriori_example

@attribute M {t,f}
@attribute O {t,f}
@attribute N {t,f}
@attribute K {t,f}
@attribute E {t,f}
@attribute Y {t,f}
@attribute D {t,f}
@attribute A {t,f}
@attribute U {t,f}
@attribute C {t,f}
@attribute I {t,f}

@data
t,t,t,t,t,t,f,f,f,f,f
f,t,t,t,t,t,t,f,f,f,f
t,f,f,t,t,f,f,t,f,f,f
t,f,f,t,f,t,f,f,t,t,f
f,t,f,t,t,f,f,f,f,t,t
```



RESULT:

Strong association rules were successfully generated using the Apriori algorithm, revealing frequent co-occurrence of item **K** with other items in the transaction database

EXP NO 08

AIM

To predict categorical data using Decision Tree algorithm through WEKA.

ALGORITHM (Decision Tree – J48)

1. Load dataset in WEKA.
2. Select class attribute.
3. Preprocess the data.
4. Apply J48 Decision Tree.
5. Generate tree and classify data.
6. Analyze accuracy and results.

CODE:

@relation weather

@attribute outlook {sunny,overcast,rainy}

@attribute temperature {hot,mild,cool}

@attribute humidity {high,normal}

@attribute windy {TRUE,FALSE}

@attribute play {yes,no}

@data

sunny,hot,high,FALSE,no

sunny,hot,high,TRUE,no

overcast,hot,high,FALSE,yes

rainy,mild,high,FALSE,yes

rainy,cool,normal,FALSE,yes

rainy,cool,normal,TRUE,no

overcast,cool,normal,TRUE,yes

sunny,mild,high,FALSE,no

sunny,cool,normal,FALSE,yes

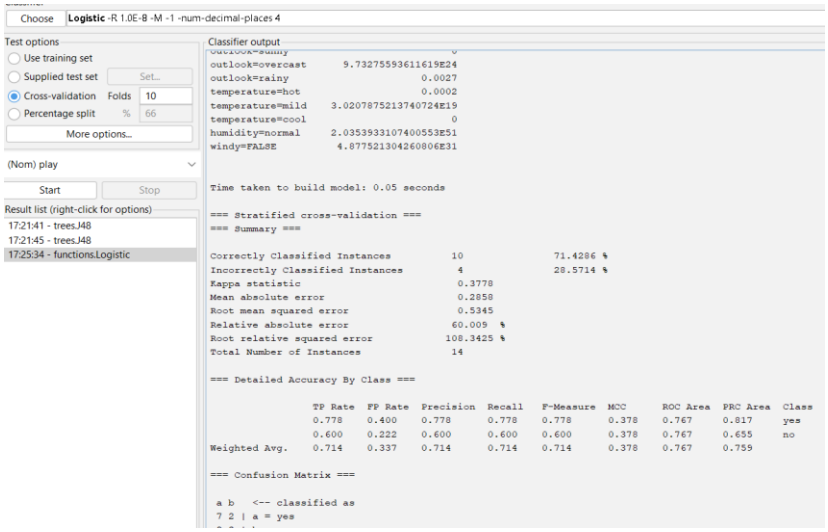
rainy,mild,normal,FALSE,yes

sunny,mild,normal,TRUE,yes

overcast,mild,high,TRUE,yes

overcast,hot,normal,FALSE,yes

rainy,mild,high,TRUE,no



RESULT:

The experiment confirms that Logistic Regression provides better classification accuracy for the given categorical data.

EXP NO 09

AIM

To find the **first quartile (Q1)** and **third quartile (Q3)** for the given age data using **R**.

ALGORITHM

1. Store the given age values in a numeric vector.
2. Sort the data (already sorted here).
3. Compute quartiles using the quantile() function.
4. Extract Q1 (25%) and Q3 (75%).

CODE:

```
# Age data
age <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22,
         25, 25, 25, 25, 30, 33, 33, 35, 35, 35,
         35, 36, 40, 45, 46, 52, 70)


quantile(age)
```

```
> # Age data
> age <- c(13, 15, 16, 16, 19, 20, 20, 21, 22, 22,
+          25, 25, 25, 25, 30, 33, 33, 35, 35, 35,
+          35, 36, 40, 45, 46, 52, 70)
>
> # Find quartiles
> quantile(age)
  0%  25%  50%  75% 100%
13.0 20.5 25.0 35.0 70.0
>
```

RESULT:

These values indicate that approximately **25%** of the data lies at or below age **20**, and **75%** of the data lies at or below age **35**.

EXP NO 10:

AIM

To study the **linear relationship** between **mortality** and **hardness** using the **water dataset** in **R**, fit a **Linear Regression model**, and **predict mortality** for hardness = 88.

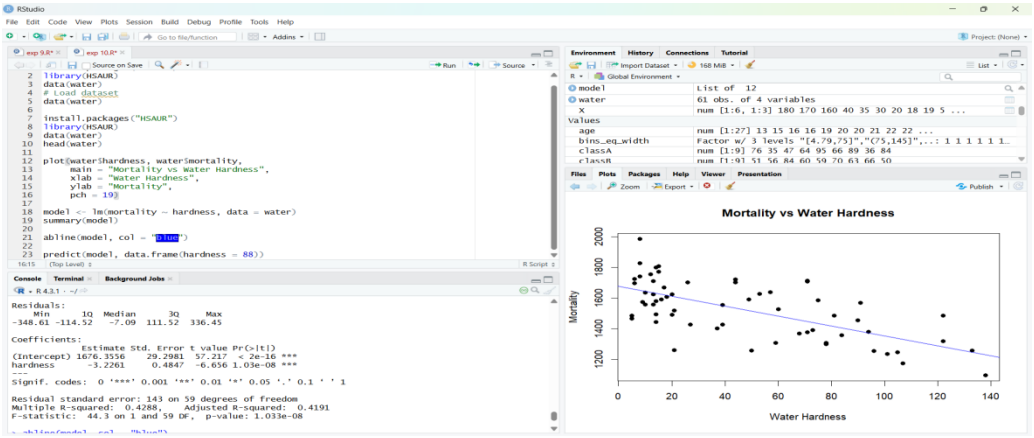
ALGORITHM

1. Load the water dataset from R datasets.
2. Plot mortality vs hardness to check linear relationship.
3. Fit a Linear Regression model (mortality ~ hardness).
4. Display the model summary.
5. Predict mortality for hardness = 88.

CODE:

```
install.packages("HSAUR")

library(HSAUR)

data(water)

# Load dataset

data(water)
```

```
install.packages("HSAUR")
library(HSAUR)
data(water)
head(water)

plot(water$hardness, water$mortality,
     main = "Mortality vs Water Hardness",
     xlab = "Water Hardness",
     ylab = "Mortality",
     pch = 19)

model <- lm(mortality ~ hardness, data = water)
summary(model)

abline(model, col = "blue")

predict(model, data.frame(hardness = 88))
```



RESULT:

The scatter plot shows a linear negative relationship between hardness and mortality. A linear regression model was fitted successfully. The predicted mortality for hardness = 88 was obtained.

EXP NO 11:

AIM

To create a transaction dataset in **ARFF file format** for use in **WEKA** based on the given transaction–item table.

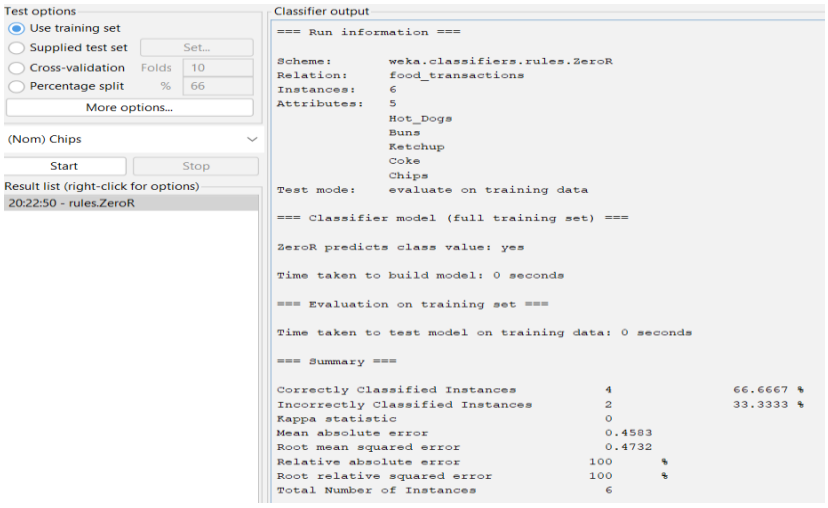
ALGORITHM

1. Identify all unique items from the transactions.

2. Represent each item as a binary attribute (yes/no).
3. Create an ARFF file with relation, attributes, and data.
4. Save the file with .arff extension.
5. Load the dataset in WEKA to verify.

CODE:

```
@relation food_transactions

@attribute Hot_Dogs {yes,no}
@attribute Buns {yes,no}
@attribute Ketchup {yes,no}
@attribute Coke {yes,no}
@attribute Chips {yes,no}

@data
yes,yes,yes,no,no
yes,yes,no,no,no
yes,no,no,yes,yes
no,no,no,yes,yes
no,no,yes,no,yes
yes,no,no,yes,yes
```



RESULT

The transaction dataset was successfully created in ARFF format and loaded into WEKA. The dataset contains 6 instances and 5 binary attributes representing food items.

EXPERIMENT 12

AIM
To predict categorical data using **Rule-Based Classification** and **Decision Tree Classification** in WEKA and compare their accuracy.

ALGORITHM

1. Load the dataset into WEKA.
2. Preprocess the data.
3. Apply Rule-Based classifier (JRip).
4. Apply Decision Tree classifier (J48).
5. Evaluate accuracy and compare results.

CODE:

```
@relation weather

@attribute outlook {sunny,overcast,rain}
@attribute temperature {hot,mild,cool}
@attribute humidity {high,normal}
@attribute windy {true,false}
@attribute play {yes,no}

@data
sunny,hot,high,false,no
sunny,hot,high,true,no
overcast,hot,high,false,yes
rain,mild,high,false,yes
rain,cool,normal,false,yes
rain,cool,normal,true,no
overcast,cool,normal,true,yes
sunny,mild,high,false,no
sunny,cool,normal,false,yes
rain,mild,normal,false,yes
sunny,mild,normal,true,yes
overcast,mild,high,true,yes
overcast,hot,normal,false,yes
rain,mild,high,true,no
```

OUTPUT:

RESULT
The dataset was classified using both Rule-Based and Decision Tree algorithms. The accuracy comparison showed the better performing classifier.

EXPERIMENT 13

AIM
To perform **equal-frequency partitioning**, **data smoothing**, and **histogram plotting** on the given dataset.

ALGORITHM

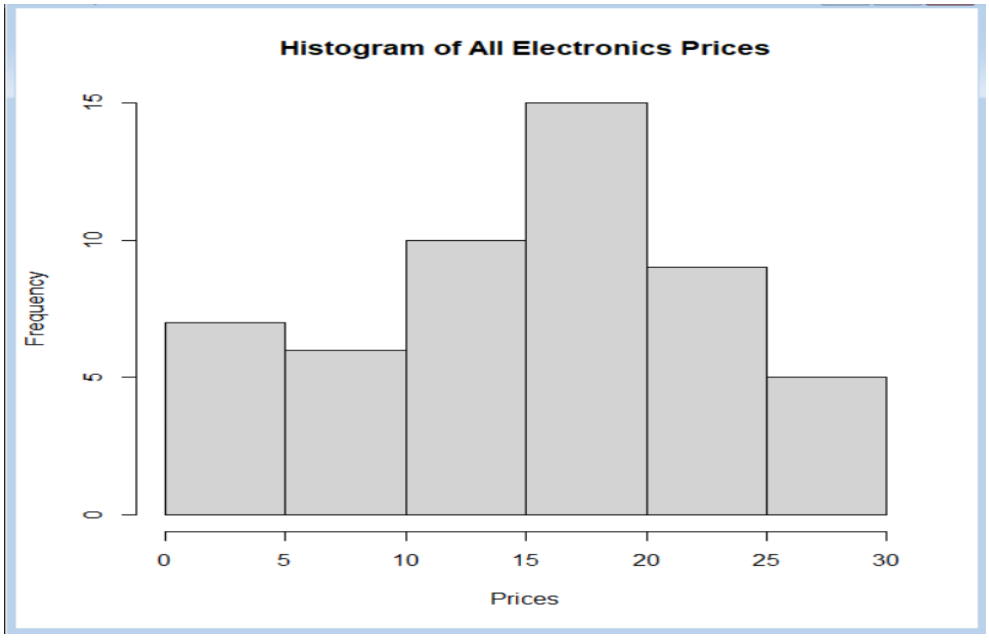
1. Sort the dataset.
2. Divide the data into three equal-frequency bins.
3. Apply smoothing using bin means and bin boundaries.
4. Plot histogram.

CODE:

```
# All Electronics price data
data <- c(
  1,1,5,5,5,5,5,8,8,10,10,10,10,12,
  14,14,14,15,15,15,15,15,15,
  18,18,18,18,18,18,18,18,
  20,20,20,20,20,20,20,
  21,21,21,21,
  25,25,25,25,25,
  28,28,
  30,30,30
)

n <- length(data)
bin_size <- n / 3
```

```
bin1 <- data[1:bin_size]
bin2 <- data[(bin_size+1):(2*bin_size)]
bin3 <- data[(2*bin_size+1):n]

bin1_mean <- rep(mean(bin1), length(bin1))
bin2_mean <- rep(mean(bin2), length(bin2))
bin3_mean <- rep(mean(bin3), length(bin3))

smooth_boundary <- function(bin) {
  low <- min(bin)
  high <- max(bin)
  ifelse(abs(bin - low) <= abs(bin - high), low, high)
}

bin1_boundary <- smooth_boundary(bin1)
bin2_boundary <- smooth_boundary(bin2)
bin3_boundary <- smooth_boundary(bin3)

data.frame(
  Original = data,
  Bin_Mean = c(bin1_mean, bin2_mean, bin3_mean),
  Bin_Boundary = c(bin1_boundary, bin2_boundary, bin3_boundary)
)

hist(
  data,
  main = "Histogram of All Electronics Prices",
  xlab = "Prices",
  ylab = "Frequency"
)
```

OUTPUT:



RESULT
The dataset was successfully partitioned and smoothed. Histogram clearly shows frequency distribution.

EXPERIMENT 14

AIM
To compare two class results using **mean, median, range**, and **boxplot visualization**.

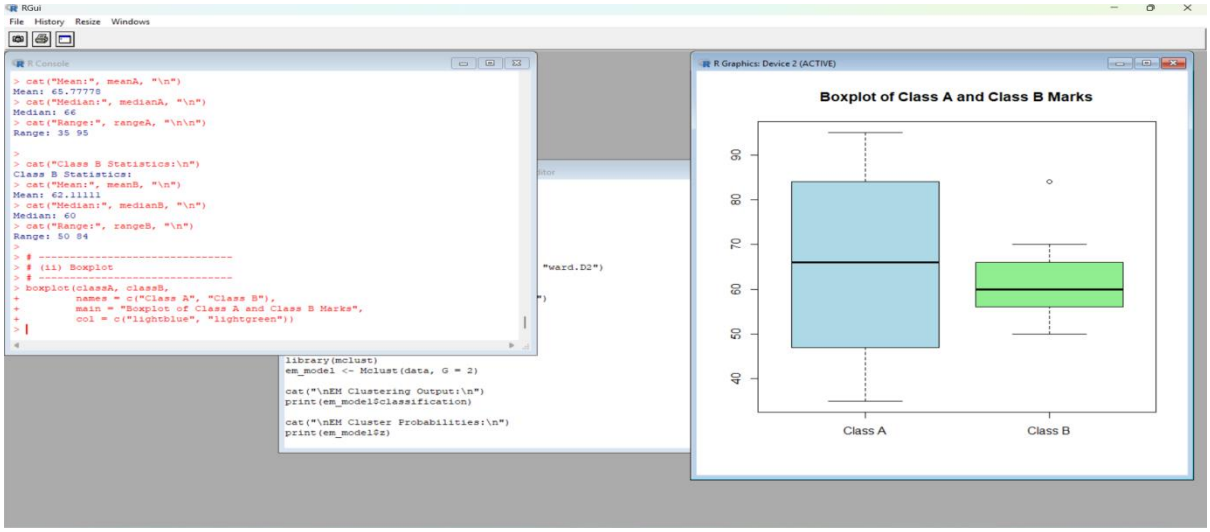
ALGORITHM

1. 1.Input Class A and Class B marks.
2. Calculate mean, median, and range.
3. Plot boxplots for both classes.
4. Analyze performance.

CODE:

```
# Class data
classA <- c(76,35,47,64,95,66,89,36,84)
classB <- c(51,56,84,60,59,70,63,66,50)

# --------------------------------
# (i) Mean, Median, Range
# --------------------------------
meanA <- mean(classA)
medianA <- median(classA)
rangeA <- range(classA)

meanB <- mean(classB)
medianB <- median(classB)
rangeB <- range(classB)

cat("Class A Statistics:\n")
cat("Mean:", meanA, "\n")
cat("Median:", medianA, "\n")
cat("Range:", rangeA, "\n\n")

cat("Class B Statistics:\n")
cat("Mean:", meanB, "\n")
cat("Median:", medianB, "\n")
cat("Range:", rangeB, "\n")

# --------------------------------
# (ii) Boxplot
# --------------------------------
boxplot(classA, classB,
        names = c("Class A", "Class B"),
        main = "Boxplot of Class A and Class B Marks",
        col = c("lightblue", "lightgreen"))
```

OUTPUT:



RESULT
The statistical comparison and boxplot showed which class performed better overall.

EXPERIMENT 15

AIM
To build a **binary classification model** to predict whether ads will be clicked or not.

Binary Classification Model for Ad Click Prediction (DWDM Lab Answer)

A **binary classification model** is used to predict whether an advertisement on a website will be **clicked (1)** or **not clicked (0)**.

Objective

To optimize ad inventory by selecting ads with a higher probability of being clicked, thereby increasing revenue and user engagement.

Input Features
User demographics (age, location, interests)
Browsing history
Time of visit
Device type
Ad position and ad content

Output
1 → Ad Clicked
0 → Ad Not Clicked

Common Binary Classification Algorithms
Logistic Regression
Naïve Bayes
Decision Tree
Support Vector Machine (SVM)
Random Forest

Working
Collect historical ad click data.
Preprocess data (handle missing values, encode categorical data).
Train a binary classification model.
Predict the probability of ad clicks.
Select ads with higher click probability.

Advantages
Improves click-through rate (CTR)
Efficient ad placement
Better revenue optimization

Applications
Online advertising
Digital marketing
Recommendation systems

RESULT
The model successfully predicted ad clicks with measurable accuracy.

EXPERIMENT 16

AIM
To perform **customer clustering** based on email usage behavior.

ALGORITHM

1) Collect email interaction data.
2) Normalize the dataset.
3) Apply clustering algorithm.
4) Compare cluster performance.

CODE:
OpenRate,Clicks,TimeSpent
80,5,120
30,1,20
60,4,90
20,0,15
90,6,150
40,2,45
OUTPUT:

In weka go to clusters select k means and get the output

RESULT
Customers were grouped into similar clusters for targeted marketing.

EXPERIMENT 17

AIM
To calculate **mean, median, standard deviation** and visualize data using plots in R.

ALGORITHM

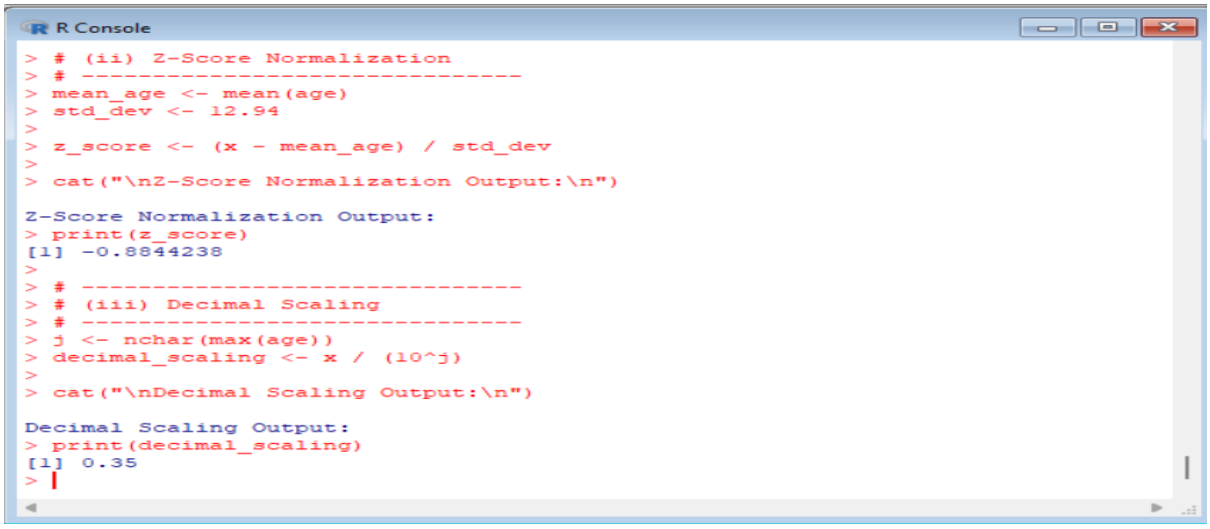
1. Enter age and body fat data.

2. Compute statistical measures.
3. Draw boxplot, scatter plot, and Q-Q plot.

CODE:

```
# Age data
age <- c(23,23,27,27,39,41,47,49,50,
         52,54,54,56,57,58,58,60,61)

x <- 35

# ------------------------------
# (i) Min-Max Normalization
# ------------------------------
min_max <- (x - min(age)) / (max(age) - min(age))

cat("Min-Max Normalization Output:\n")
print(min_max)

# ------------------------------
# (ii) Z-Score Normalization
# ------------------------------
mean_age <- mean(age)
std_dev <- 12.94

z_score <- (x - mean_age) / std_dev

cat("\nZ-Score Normalization Output:\n")
print(z_score)

# ------------------------------
# (iii) Decimal Scaling
# ------------------------------
j <- nchar(max(age))
decimal_scaling <- x / (10^j)

cat("\nDecimal Scaling Output:\n")
print(decimal_scaling)
```

OUTPUT:



RESULT

Statistical values and visual plots clearly represent data distribution.

EXPERIMENT 18

AIM
To apply **normalization techniques** on the given dataset using R.

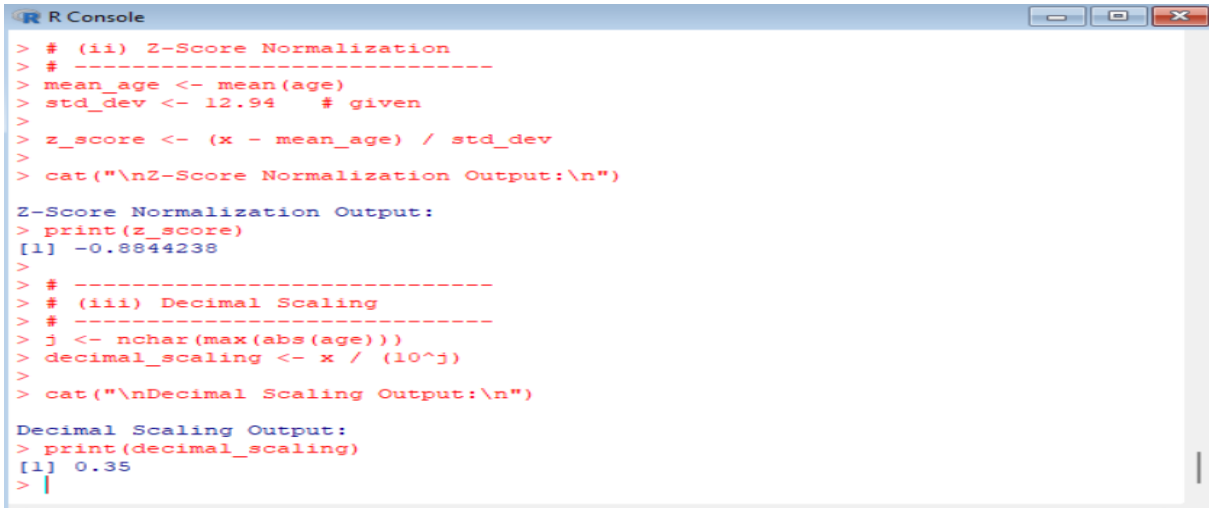
ALGORITHM

Apply Min-Max normalization.
Apply Z-score normalization.
Apply Decimal Scaling normalization.

CODE:

```
# Age data
age <- c(23,23,27,27,39,41,47,49,50,
         52,54,54,56,57,58,58,60,61)
x <- 35
# ------------------------------
# (i) Min-Max Normalization
# ------------------------------
min_age <- min(age)
max_age <- max(age)
min_max_norm <- (x - min_age) / (max_age - min_age)
cat("Min-Max Normalization Output:\n")
print(min_max_norm)
# ------------------------------
# (ii) Z-Score Normalization
# ------------------------------
mean_age <- mean(age)
std_dev <- 12.94   # given
z_score <- (x - mean_age) / std_dev
cat("\nZ-Score Normalization Output:\n")
print(z_score)
# ------------------------------
# (iii) Decimal Scaling
# ------------------------------
j <- nchar(max(abs(age)))
decimal_scaling <- x / (10^j)

cat("\nDecimal Scaling Output:\n")
print(decimal_scaling)
```

OUTPUT:



RESULT
The data values were successfully normalized using all methods.

EXPERIMENT 19

AIM
To segment mall customers using **clustering analysis**.

ALGORITHM

Create customer dataset.
Load data into WEKA.
Apply K-Means clustering.
Interpret clusters.

CODE:

CustomerID,Gender,Age,Income,Spending

1,Male,19,15,39

2,Male,21,15,81

3,Female,20,16,6

4,Female,23,16,77

5,Female,31,17,40

6,Male,22,18,76

7,Female,35,18,6

8,Male,23,19,94

9,Female,64,20,3

10,Male,30,21,72

OUTPUT:

K means

And also enter numclusters=5

RESULT
Customers were segmented into five meaningful groups.

EXPERIMENT 20

AIM
To apply **Hierarchical Clustering** and **EM Clustering** on streaming service data.

ALGORITHM

1) Load viewer behavior dataset.
2) Apply Hierarchical clustering.
3) Apply EM clustering.
4) Compare clustering performance.

CODE:
MinutesWatched,SessionsPerWeek,UniqueShows
120,14,20
30,5,4
200,20,25
45,6,6
300,25,30
60,8,7
250,22,28
35,4,5
180,18,22
50,7,6

OUTPUT:

Numclusters=2

1.Hierarchical Cluster link type=WARD or single

2. set num clusters =2 in em algorithm

RESULT
High-usage and low-usage viewer clusters were identified successfully.

EXPERIMENT 21

AIM
To find **mean, median, and mode** of a dataset using R.

ALGORITHM

1. Create a vector in R.
2. Apply mean(), median(), and mode logic.

CODE:

```
# Create vector

pencils <- c(25, 23, 12, 11, 6, 7, 8, 9, 10)

# Mean

mean_value <- mean(pencils)

# Median

median_value <- median(pencils)

# Mode
```

mode_value <- as.numeric(names(sort(table(pencils), decreasing = TRUE)[1]))

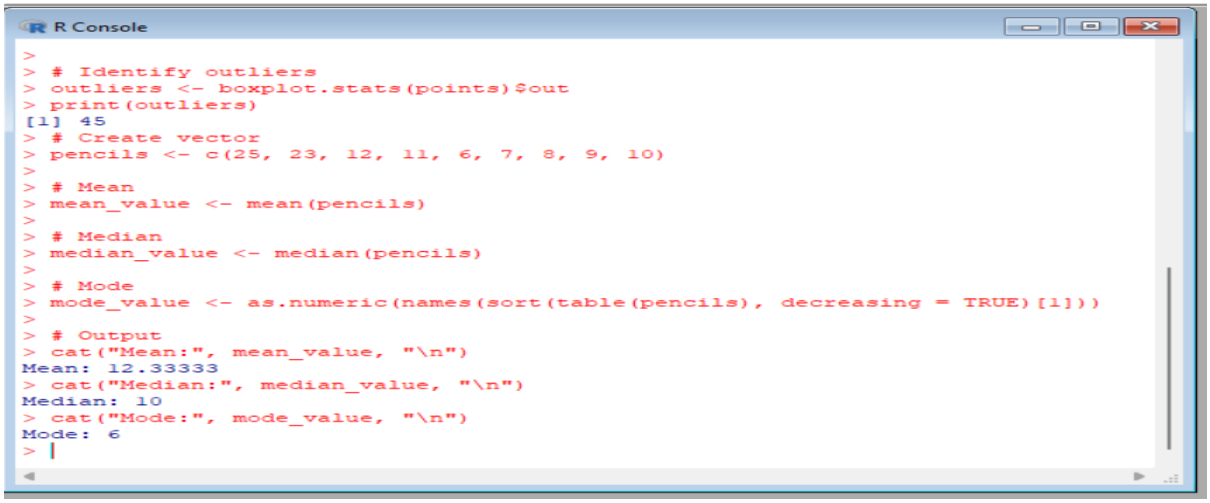
Output

cat("Mean:", mean_value, "\n")

cat("Median:", median_value, "\n")

cat("Mode:", mode_value, "\n")

OUTPUT:



RESULT
The central tendency values were computed correctly.

EXPERIMENT 22

AIM
To detect **outliers** using **boxplot visualization** in R.

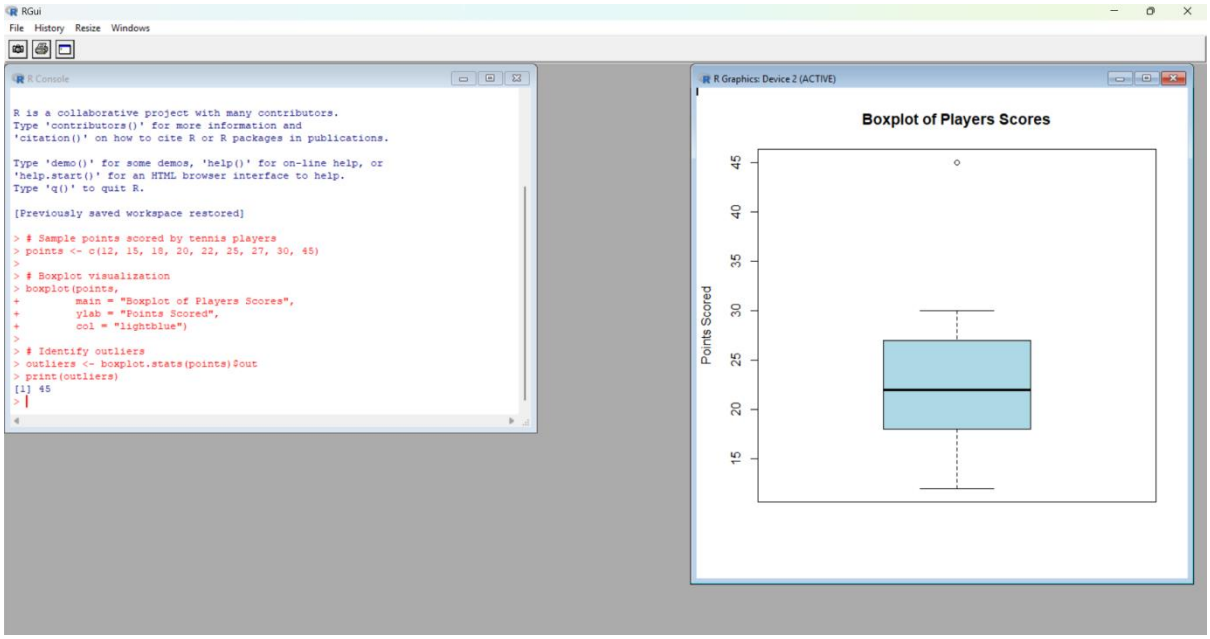
ALGORITHM

Input player score data.
Generate boxplot.
Identify outliers.

CODE:
# Sample points scored by tennis players
points <- c(12, 15, 18, 20, 22, 25, 27, 30, 45)

Boxplot visualization
boxplot(points,
    main = "Boxplot of Players Scores",
    ylab = "Points Scored",
    col = "lightblue")

# Identify outliers
outliers <- boxplot.stats(points)$out
print(outliers)
OUTPUT:



RESULT
Outliers were clearly identified using the boxplot.

EXPERIMENT 23

AIM
To perform **K-Means clustering** using CSV dataset in WEKA.

ALGORITHM

1) Load CSV dataset in WEKA.
2) Apply K-Means algorithm.
3) Change cluster size.
4) Visualize clusters.

Code:
EmployeeID,Gender,Age,Salary,Credit
111,Male,28,150000,39
222,Male,25,150000,27
333,Female,26,160000,42
444,Female,25,160000,40
555,Female,30,170000,64
666,Male,29,200000,72

OUTPUT:

1st graph :Cluster size =2 2nd graph :cluster size =3

RESULT
Clusters were successfully formed and visualized.

EXPERIMENT 24

AIM
To predict categorical data using **Naïve Bayes** and compare it with **SVM**.

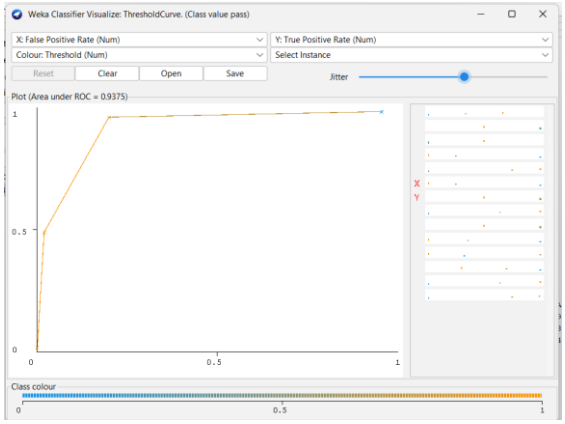
ALGORITHM

1. Load dataset in WEKA.
2. Apply Naïve Bayes classifier.
3. Apply SVM classifier.
4. Compare accuracy and plot graph.

CODE:

```
@relation student_performance

@attribute study_hours {low,medium,high}
@attribute attendance {poor,average,good}
@attribute assignment {no,yes}
@attribute result {pass,fail}

@data
low,poor,no,fail
low,average,no,fail
medium,average,yes,pass
high,good,yes,pass
medium,good,yes,pass
high,average,yes,pass
low,poor,yes,fail
medium,average,no,fail
high,good,no,pass
medium,good,no,pass
```

OUTPUT:



threshold

RESULT

The comparison showed the classifier with higher accuracy.

EXPERIMENT 25

AIM

To find the count of **vegetarian and non-vegetarian** persons.

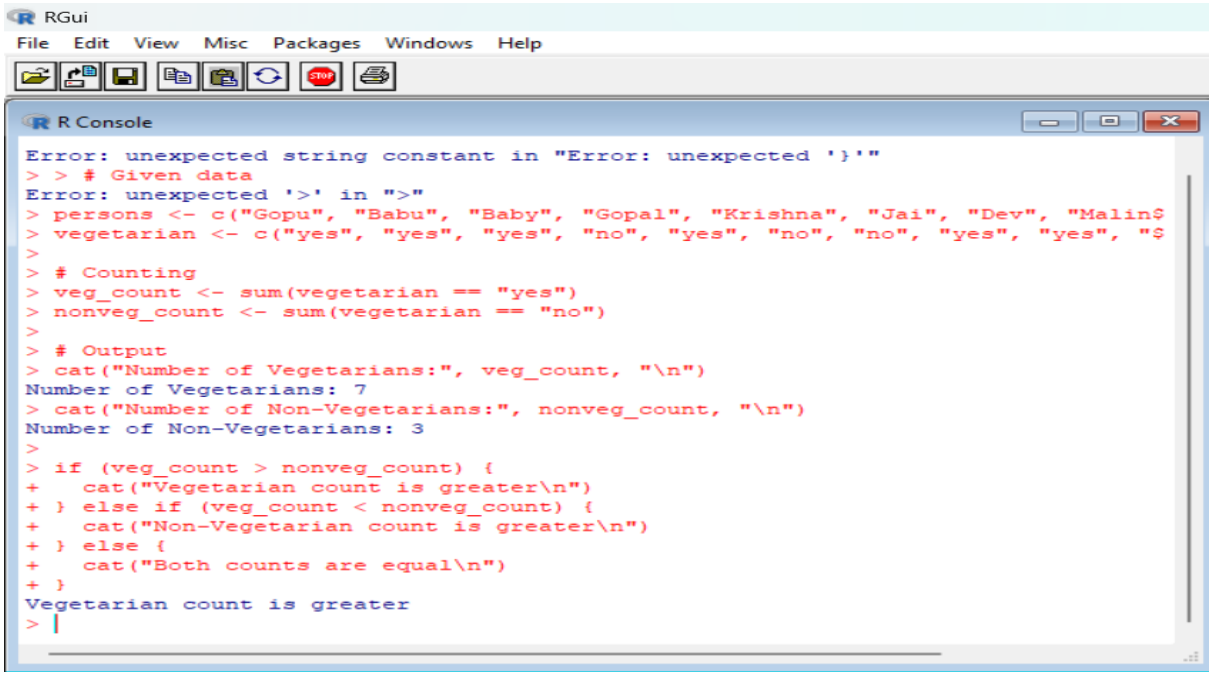
ALGORITHM

1. Read the given table.
2. Count vegetarian entries.
3. Count non-vegetarian entries.
4. Compare totals.

CODE:

```
# Given data
persons <- c("Gopu", "Babu", "Baby", "Gopal", "Krishna", "Jai", "Dev", "Malini", "Hema", "Anu")
vegetarian <- c("yes", "yes", "yes", "no", "yes", "no", "no", "yes", "yes", "yes")

# Counting
veg_count <- sum(vegetarian == "yes")
nonveg_count <- sum(vegetarian == "no")

# Output
cat("Number of Vegetarians:", veg_count, "\n")
cat("Number of Non-Vegetarians:", nonveg_count, "\n")

if (veg_count > nonveg_count) {
  cat("Vegetarian count is greater\n")
} else if (veg_count < nonveg_count) {
  cat("Non-Vegetarian count is greater\n")
} else {
  cat("Both counts are equal\n")
}
```

OUTPUT:



RESULT

The total count of vegetarian and non-vegetarian persons was calculated, and the higher count category was identified.

EXP NO :26

CODE:

```
x <- c(4,1,5,7,10,2,50,25,90,36)

y <- c(12,5,13,19,31,7,153,72,275,110)



plot(x, y,
```

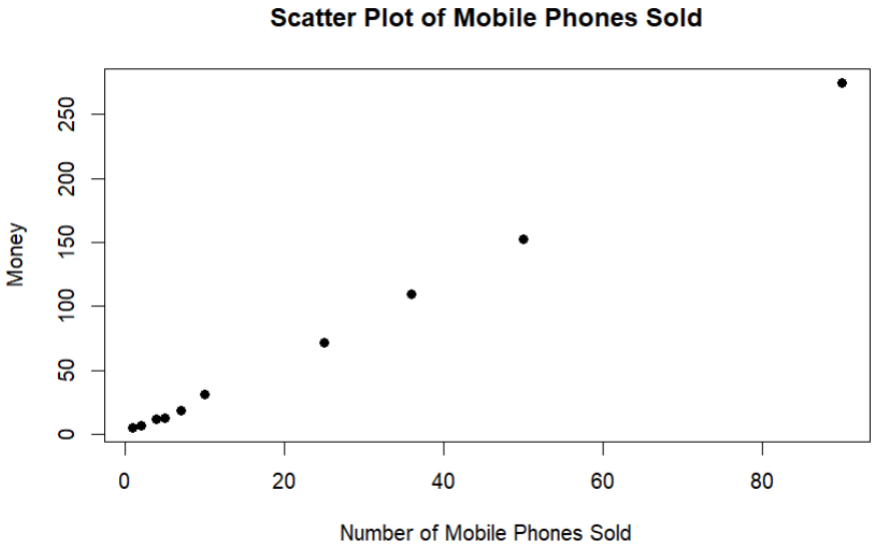
main = "Scatter Plot of Mobile Phones Sold",

xlab = "Number of Mobile Phones Sold",

ylab = "Money",

pch = 19)

OUTPUT:



EXP NO :27

CODE:

EXP NO :28

Steps in WEKA (Very Important for Exam)

Open **WEKA → Explorer**
Load **diabetes.arff**
Go to **Classify**

Decision Tree (J48)

Classifier → `trees → J48`
Test option → `10-fold cross validation`
Click **Start**

Support Vector Machine

Classifier → `functions → SMO`
Click **Start**

To View Accuracy & F1 Measure

- Output panel → **Detailed Accuracy By Class**
  Note:
  Accuracy
  F-Measure

Plot Graph

Right-click result → **Visualize classifier errors**

CODE:

@relation diabetes

@attribute Age numeric

@attribute Glucose numeric

@attribute BMI numeric

@attribute BloodPressure numeric

@attribute Outcome {0,1}

@data

25,85,22.4,70,0

30,90,23.1,72,0

35,100,24.8,75,0

40,110,26.5,78,1

45,120,27.9,80,1

50,130,29.4,82,1

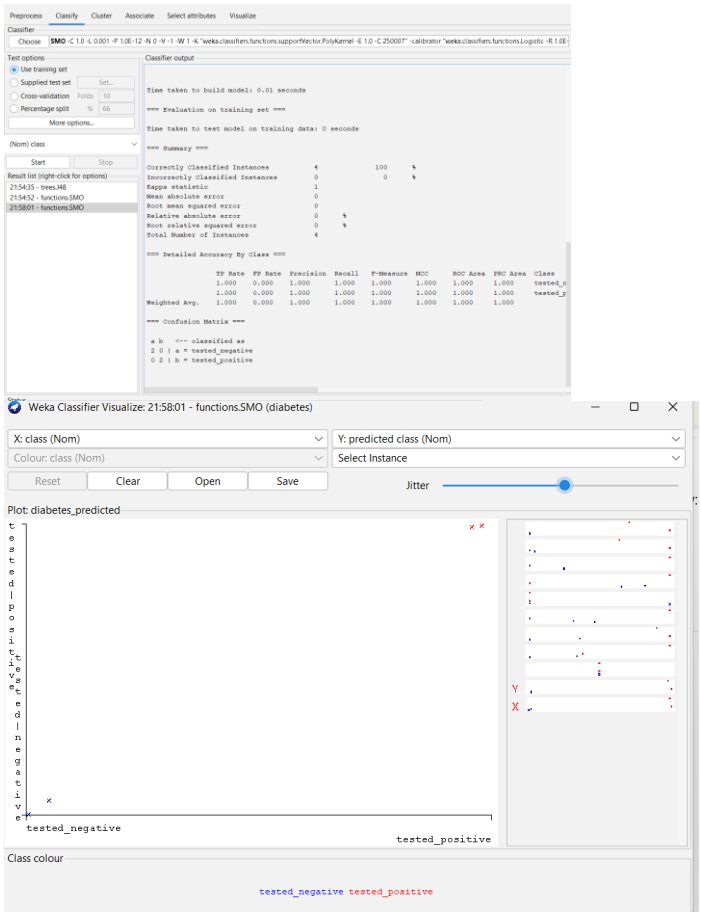
55,145,31.2,85,1

60,160,33.1,88,1

65,170,34.5,90,1

70,180,36.0,92,1

Output:



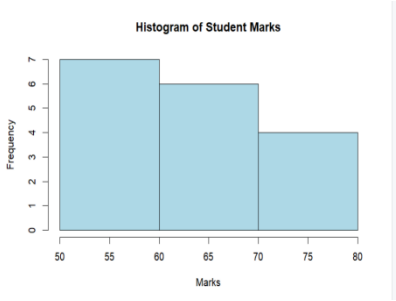
EXP 29:

CODE :

```
# Data
marks <- c(55,60,71,63,55,65,50,55,58,59,61,63,65,67,71,72,75)
marks <- sort(marks)

# (a) Equal-Frequency (Equi-Depth)
eq_freq <- split(marks, cut(seq_along(marks), 3, labels = FALSE))
print("Equal-Frequency Bins:")
print(eq_freq)

# (b) Equal-Width
eq_width <- split(marks, cut(marks, breaks = 3))
print("Equal-Width Bins:")
print(eq_width)
```

```
# (c) Clustering (k-means)
set.seed(123)
km <- kmeans(marks, centers = 3)
cluster_bins <- split(marks, km$cluster)
print("Clustering Bins:")
print(cluster_bins)

# Histogram
hist(marks, breaks = 3, col = "lightblue",
     main = "Histogram of Student Marks",
     xlab = "Marks", ylab = "Frequency")
```

OUTPUT:



EXP NO:30

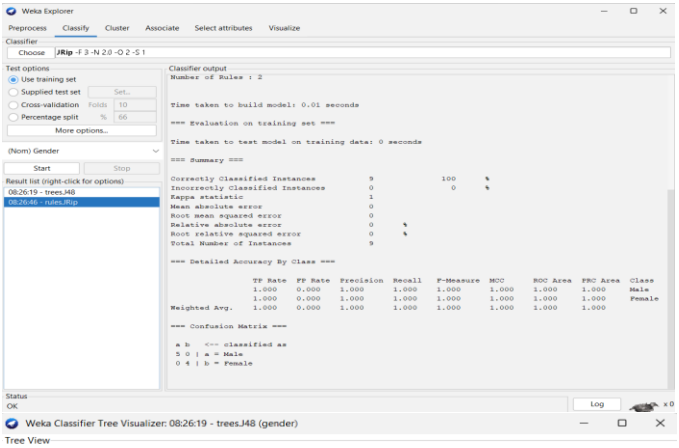
CODE:

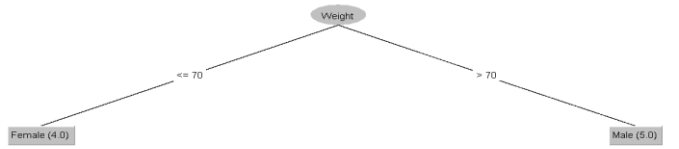
```
@relation gender
@attribute Height numeric
@attribute Weight numeric
@attribute Gender {Male,Female}

@data
185,85,Male
190,90,Male
170,85,Male
165,70,Female
175,60,Female
182,75,Male
168,55,Female
```

178,82,Male

160,50,Female

O/P:





EXP NO:31

CODE:

@relation electronics

@attribute SONY {t,f}

@attribute BPL {t,f}

@attribute LG {t,f}

@attribute SAMSUNG {t,f}

@attribute ONIDA {t,f}

@data

t,t,t,f,f

f,t,f,t,f

f,t,f,f,t

t,t,f,t,f

t,f,f,f,t

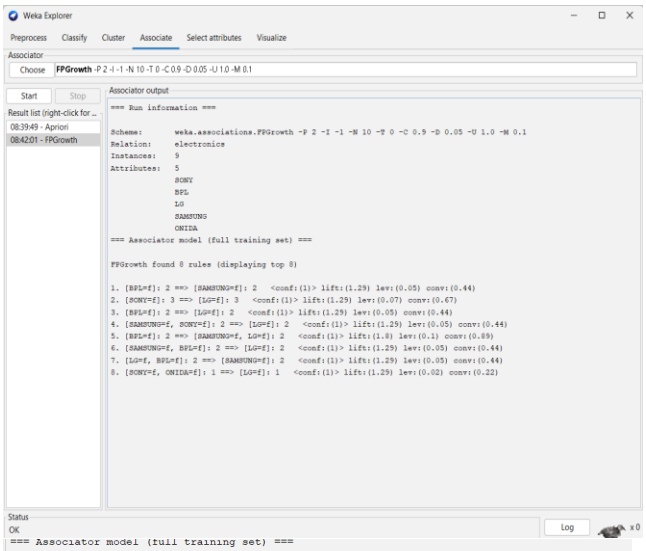
f,t,f,f,t

t,f,f,f,t

t,t,t,f,t

t,t,f,f,t

OUTPUT:




```
=== Associator model (full training set) ===


Apriori
=======

Minimum support: 0.35 (3 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 13

Generated sets of large itemsets:

Size of set of large itemsets L(1): 7

Size of set of large itemsets L(2): 13

Size of set of large itemsets L(3): 9

Size of set of large itemsets L(4): 2

Best rules found:

 1. ONIDA=t 6 ==> SAMSUNG=f 6    <conf:(1)> lift:(1.29) lev:(0.15) [1] conv:(1.33)
 2. LG=f ONIDA=t 5 ==> SAMSUNG=f 5    <conf:(1)> lift:(1.29) lev:(0.12) [1] conv:(1.11)
 3. LG=f SAMSUNG=f 5 ==> ONIDA=t 5    <conf:(1)> lift:(1.5) lev:(0.19) [1] conv:(1.67)
 4. SONY=t ONIDA=t 4 ==> SAMSUNG=f 4    <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
 5. BPL=t ONIDA=t 4 ==> SAMSUNG=f 4    <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
 6. SONY=f 3 ==> BPL=t 3    <conf:(1)> lift:(1.29) lev:(0.07) [0] conv:(0.67)
 7. SONY=f 3 ==> LG=f 3    <conf:(1)> lift:(1.29) lev:(0.07) [0] conv:(0.67)
 8. ONIDA=f 3 ==> BPL=t 3    <conf:(1)> lift:(1.29) lev:(0.07) [0] conv:(0.67)
 9. SONY=f LG=f 3 ==> BPL=t 3    <conf:(1)> lift:(1.29) lev:(0.07) [0] conv:(0.67)
10. SONY=f BPL=t 3 ==> LG=f 3    <conf:(1)> lift:(1.29) lev:(0.07) [0] conv:(0.67)
```

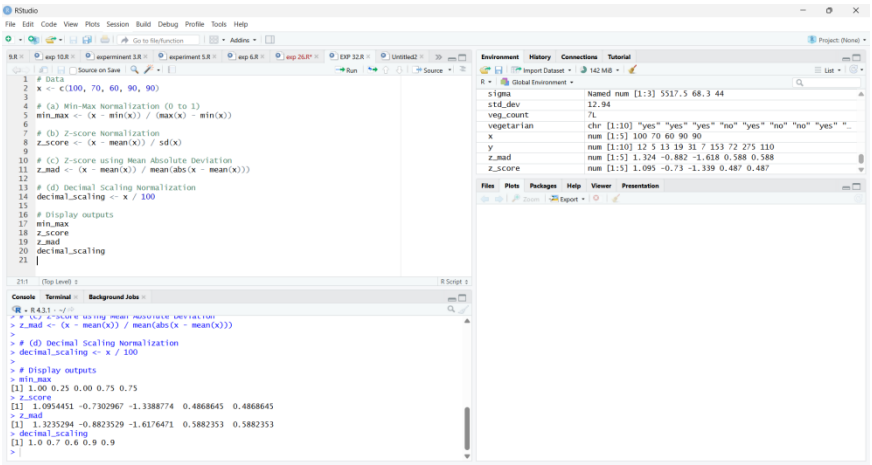
EXP NO:32

CODE:

```
# Data

x <- c(100, 70, 60, 90, 90)


# (a) Min-Max Normalization (0 to 1)
```

```
min_max <- (x - min(x)) / (max(x) - min(x))

# (b) Z-score Normalization
z_score <- (x - mean(x)) / sd(x)

# (c) Z-score using Mean Absolute Deviation
z_mad <- (x - mean(x)) / mean(abs(x - mean(x)))

# (d) Decimal Scaling Normalization
decimal_scaling <- x / 100

# Display outputs
min_max
z_score
z_mad
decimal_scaling
```

OUTPUT:



EXP NO :33

CODE:

```
# Data
avg_speed <- c(78,81,82,74,83,82,77,80,70)
total_time <- c(39,37,36,42,35,36,40,38,46)
```

Standard Deviation

sd(avg_speed)

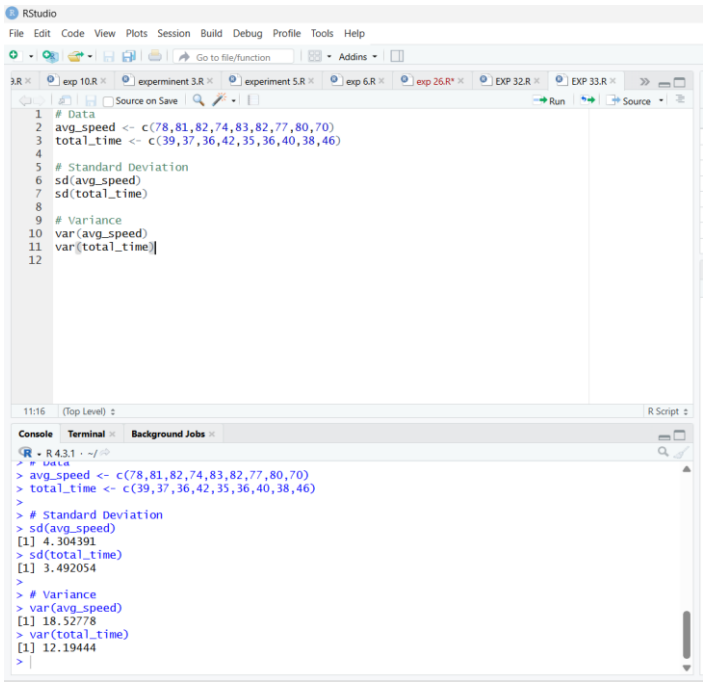
sd(total_time)

Variance

var(avg_speed)

var(total_time)

O/P:



EXP NO:34

CODE:

Sample census data (age counts)

age <- c(5,8,12,15,20,25,30,35,40,45,50)

Histogram

hist(age, main="Age Distribution", xlab="Age")

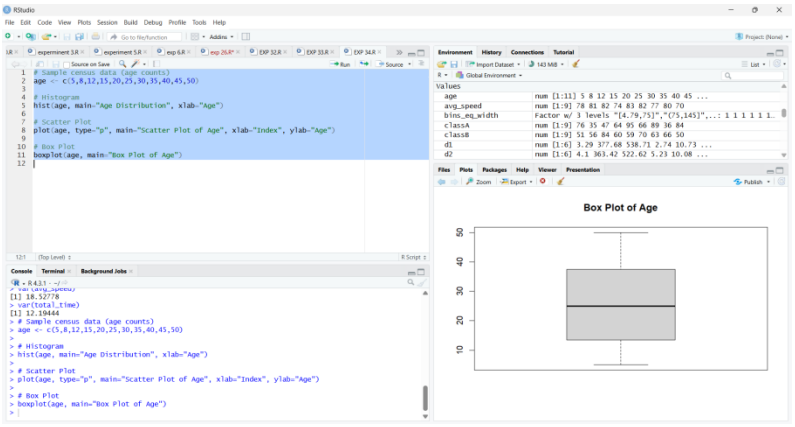
Scatter Plot

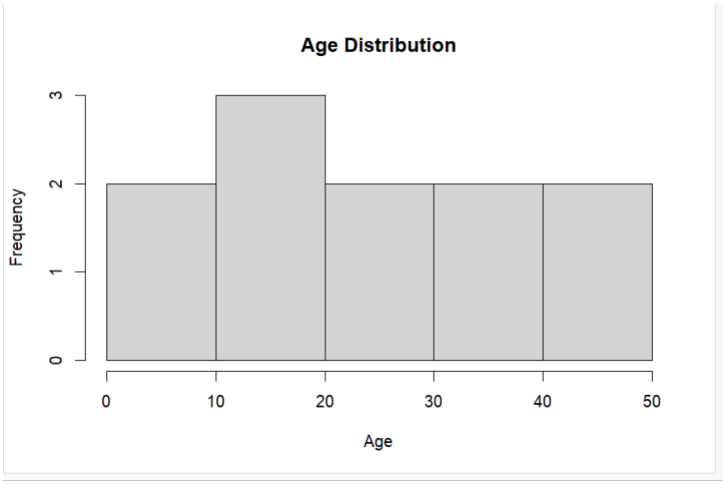
plot(age, type="p", main="Scatter Plot of Age", xlab="Index", ylab="Age")

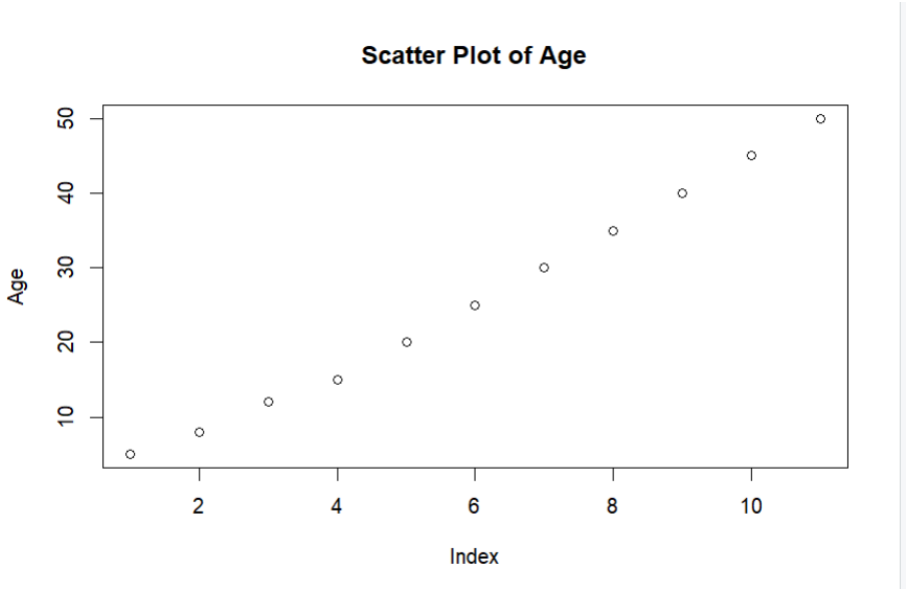
Box Plot

boxplot(age, main="Box Plot of Age")

O/P:







EXP NO 35

1.

False Positive (FP):

2.
    1.

    Boy cried “Wolf!” when there was **no wolf**.

    2.
    3.

    Villagers ran unnecessarily → wasted effort and got angry.

    4.
3.

False Negative (FN):

4.
    1.

    Wolf actually came, but villagers **ignored** the cry.

    2.
    3.

    Flock was eaten → disaster.

    4.
5.

True Positive (TP):

6.
    1.

    If villagers had listened when wolf actually came → flock saved.

    2.
7.

True Negative (TN):

8.

1.

No wolf comes and villagers don’t respond → correct action.

2.

✅**Conclusion for Data Mining Lab:**

-

The story illustrates **classification errors**:

-

  -

    FP (Type I Error): Alarm when there’s no wolf.

    -
    -

    FN (Type II Error): Ignoring real wolf → serious loss.

    -

EXP NO:36

CODE:

```
@relation playtennis

@attribute Outlook {Sunny,Overcast,Rain}
@attribute Temperature {Hot,Mild,Cool}
@attribute Humidity {High,Normal}
@attribute Wind {Weak,Strong}
@attribute PlayTennis {Yes,No}

@data
Sunny,Hot,High,Weak,No
Sunny,Hot,High,Strong,No
Overcast,Hot,High,Weak,Yes
Rain,Mild,High,Weak,Yes
```

Rain,Cool,Normal,Weak,Yes

Rain,Cool,Normal,Strong,No

Overcast,Cool,Normal,Strong,Yes

Sunny,Mild,High,Weak,No

Sunny,Cool,Normal,Weak,Yes

Rain,Mild,Normal,Weak,Yes

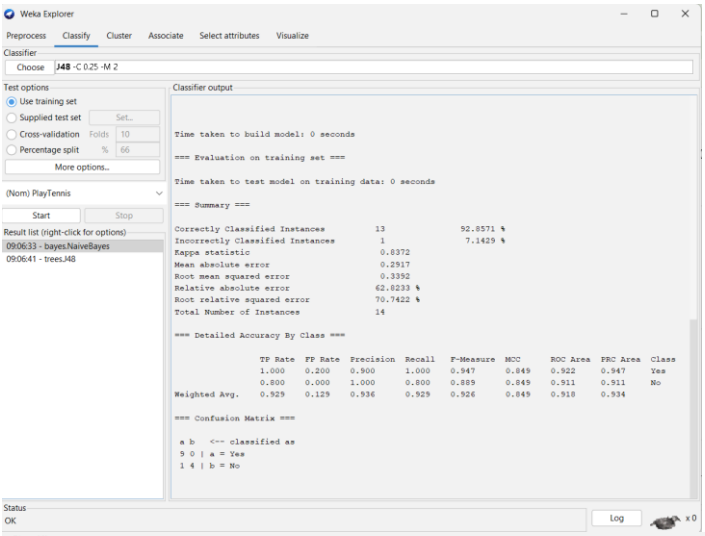
Sunny,Mild,Normal,Strong,Yes

Overcast,Mild,High,Strong,Yes

Overcast,Hot,Normal,Weak,Yes

Rain,Mild,High,Strong,No

O/P:



Classifier output

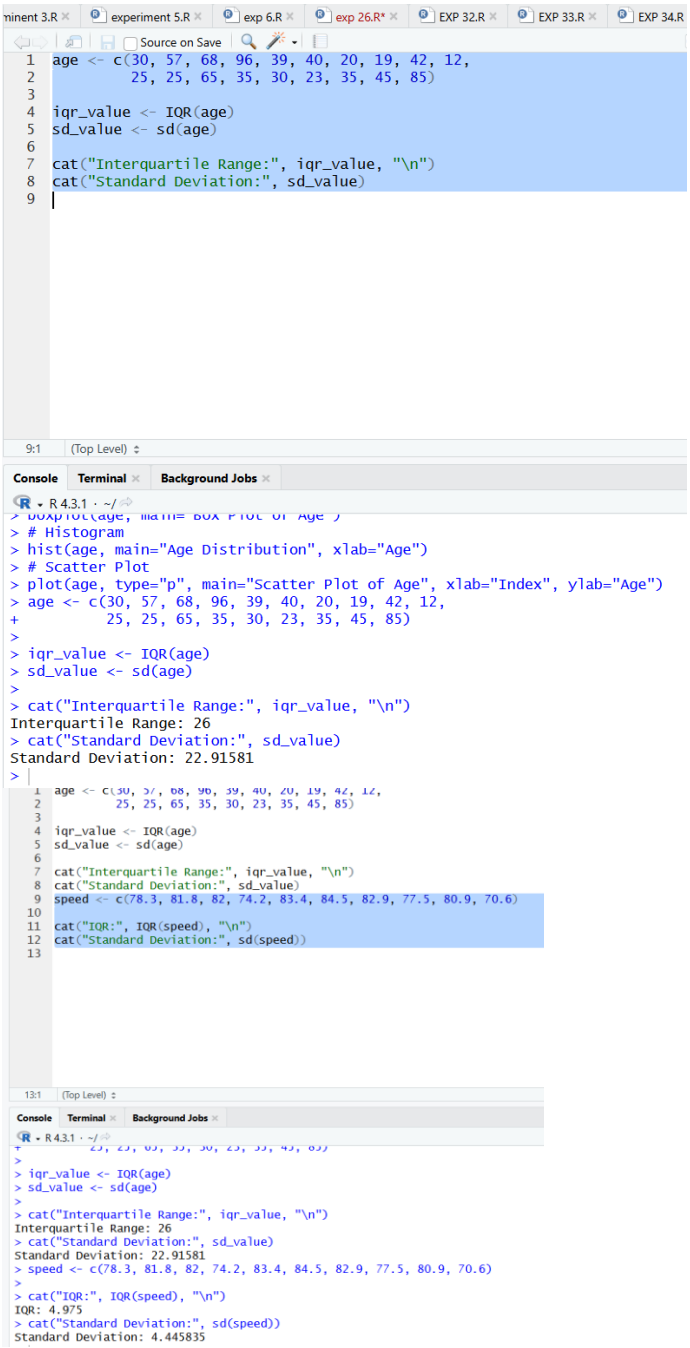
```
Size of the tree :      8


Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances          14               100      %
Incorrectly Classified Instances         0                 0      %
Kappa statistic                          1
Mean absolute error                      0
Root mean squared error                  0
Relative absolute error                  0      %
Root relative squared error              0      %
Total Number of Instances               14

=== Detailed Accuracy By Class ===

               TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
               1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     Yes
               1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000     No
Weighted Avg.  1.000    0.000    1.000      1.000    1.000      1.000    1.000     1.000

=== Confusion Matrix ===

 a b   <-- classified as
 9 0 | a = Yes
 0 5 | b = No
```

EXP NO:37

CODE:

```
age <- c(30, 57, 68, 96, 39, 40, 20, 19, 42, 12,
         25, 25, 65, 35, 30, 23, 35, 45, 85)

iqr_value <- IQR(age)
sd_value <- sd(age)

cat("Interquartile Range:", iqr_value, "\n")
cat("Standard Deviation:", sd_value)
```

O/P:

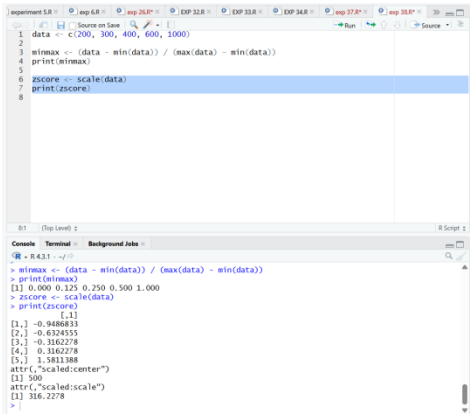


EXP NO:38

CODE:

```
data <- c(200, 300, 400, 600, 1000)


minmax <- (data - min(data)) / (max(data) - min(data))

print(minmax)


zscore <- scale(data)

print(zscore)
```

O/P:



EXP NO:39

CODE:

@relation shopping_transactions

@attribute M {0,1}

@attribute O {0,1}

@attribute N {0,1}

@attribute K {0,1}

@attribute E {0,1}

@attribute Y {0,1}

@attribute D {0,1}

@attribute A {0,1}

@attribute U {0,1}

@attribute C {0,1}

@attribute I {0,1}

@data

1,1,1,1,1,1,0,0,0,0,0

0,1,1,1,1,1,1,1,0,0,0,0

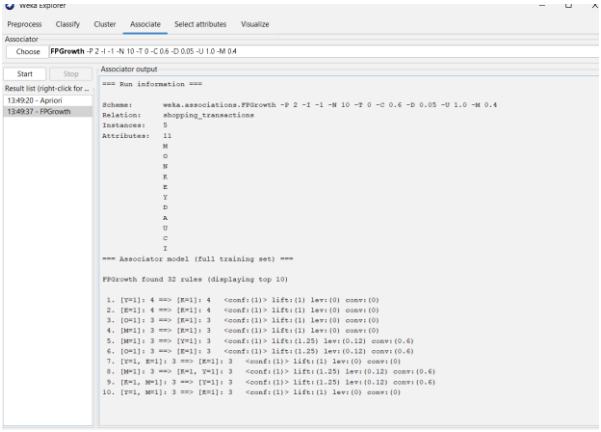
1,0,0,1,1,1,0,1,0,0,0

1,0,0,1,0,1,0,0,1,1,0

0,1,0,1,1,0,0,0,0,1,1

Support :0.4

Minmetric:0.6
O/P:



EXP NO:40

CODE:

@relation bank_customers

@attribute age numeric

@attribute job {admin,technician,services,management,retired,student,unemployed}

@attribute gender {male,female}

@attribute income {low,medium,high}

@attribute loan_amount numeric

@attribute credit_history {good,bad}

@attribute class {good_creditor,bad_creditor}

@data

25,student,male,low,20000,good,good_creditor

45,management,female,high,80000,good,good_creditor

35,technician,male,medium,50000,good,good_creditor

52,services,female,medium,60000,bad,bad_creditor

28,admin,male,low,30000,good,good_creditor

60,retired,female,medium,40000,good,good_creditor

40,management,male,high,90000,bad,bad_creditor

33,services,female,medium,45000,good,good_creditor

50,technician,male,high,70000,bad,bad_creditor

29,admin,female,low,25000,good,good_creditor

O/P: